

Terraform Hands-On Lab

State Locking · Lock File · Destroy Protection

Prerequisites: AWS CLI configured with credentials, Terraform installed (v1.5+), an AWS account/sandbox you can safely test in.

Lab 1 — State Locking (S3 backend + DynamoDB)

Goal: Prove that Terraform blocks a second concurrent apply while one is already running, using a remote backend with locking enabled.

Step 1 — Create the backend resources

```
# S3 bucket for remote state
aws s3api create-bucket \
  --bucket my-tf-state-lab-<yourname> \
  --region us-east-1

aws s3api put-bucket-versioning \
  --bucket my-tf-state-lab-<yourname> \
  --versioning-configuration Status=Enabled

# DynamoDB table for locking
aws dynamodb create-table \
  --table-name terraform-locks-lab \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

Step 2 — Configure backend in main.tf

```
terraform {
  backend "s3" {
    bucket     = "my-tf-state-lab-<yourname>"
    key        = "lab/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "terraform-locks-lab"
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "lab_vm" {
  ami           = "ami-0c101f26f147fa7fd" # verify current AMI for your region
  instance_type = "t2.micro"
  tags = {
    Name = "lab-locking-demo"
  }
}
```

Step 3 — Initialize and apply

```
terraform init
terraform apply
```

While apply is running, open a **second terminal** and run **terraform apply** again.

Step 4 — Observe the lock

```
Error: Error acquiring the state lock
```

Lock Info:

```
ID:          xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx
Path:        my-tf-state-lab-<yourname>/lab/terraform.tfstate
Operation:   OperationTypeApply
Who:         yourname@hostname
Version:     1.x.x
Created:     2026-07-13 ...
```

Step 5 — Confirm the lock releases

Wait for the first apply to finish, then re-run the second one — it should now succeed.

Step 6 — (Optional) force-unlock demo

```
terraform force-unlock <LOCK_ID>
```

■ Only do this if certain no other process is actually running — force-unlocking a genuinely active operation can corrupt state.

Cleanup

```
terraform destroy
aws dynamodb delete-table --table-name terraform-locks-lab
aws s3 rb s3://my-tf-state-lab-<yourname> --force
```

Lab 2 — Lock File (.terraform.lock.hcl)

Goal: See how the lock file pins provider versions, and what happens when it's missing, present, or deliberately upgraded.

Step 1 — Init and inspect

```
mkdir tf-lockfile-lab && cd tf-lockfile-lab
cat > main.tf << 'EOF'
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
EOF

terraform init
cat .terraform.lock.hcl
```

You'll see something like:

```
provider "registry.terraform.io/hashicorp/aws" {
  version      = "5.42.0"
  constraints = "~> 5.0"
  hashes = [
    "h1:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx=",
    ...
  ]
}
```

Step 2 — Simulate a team member on a different day

```
rm -rf .terraform
terraform init
```

Confirm the same version installs again — the lock file forced consistency, even though ~> 5.0 would otherwise allow any 5.x version.

Step 3 — Deliberately upgrade

```
terraform init -upgrade
cat .terraform.lock.hcl
```

Compare version numbers before/after — this is the only command that changes the lock file's pinned version.

Step 4 — Break it on purpose

Manually edit .terraform.lock.hcl and change a hash value to something invalid, then run:

```
terraform init
```

You should get a checksum mismatch error — this demonstrates the tamper/integrity protection the lock file provides.

Step 5 — Restore

```
rm -rf .terraform .terraform.lock.hcl  
terraform init
```

Takeaway

Lock file committed to git → whole team + CI/CD get identical provider binaries. Only -upgrade changes it intentionally. It has no relation to state locking or to destroy protection — a separate concern entirely.

Lab 3 — Restricting terraform destroy in Prod

Goal: Test three layers of destroy protection: prevent_destroy, an IAM deny policy, and a manual-approval CI-style gate.

Part A — prevent_destroy lifecycle rule

```
resource "aws_instance" "prod_like" {
  ami          = "ami-0c101f26f147fa7fd"
  instance_type = "t2.micro"
  tags = {
    Name = "prod-protected-demo"
  }

  lifecycle {
    prevent_destroy = true
  }
}
```

```
terraform init
terraform apply
terraform destroy
```

Expected result:

```
Error: Instance cannot be destroyed

on main.tf line X:
resource "aws_instance" "prod_like" {

Resource aws_instance.prod_like has lifecycle.prevent_destroy
set, but the plan calls for this resource to be destroyed.
```

To tear it down for cleanup, remove the prevent_destroy block first, apply to update the lifecycle, then destroy.

Part B — IAM Deny Policy (enforced by AWS itself)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": [
        "ec2:TerminateInstances",
        "rds>DeleteDBInstance"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/Environment": "prod"
        }
      }
    }
  ]
}
```

```
aws iam create-policy \
  --policy-name deny-prod-destroy \
  --policy-document file:///deny-prod-destroy.json

aws iam attach-role-policy \
  --role-name <your-terraform-execution-role> \
  --policy-arn arn:aws:iam::<account-id>:policy/deny-prod-destroy
```

Tag a test instance Environment = prod, then try terraform destroy. Terraform will attempt the API call, but AWS itself rejects it — independent of what your .tf config or lifecycle blocks say.

Part C — Manual approval gate (CI/CD simulation)

```
#!/bin/bash
terraform plan -out=tfplan
echo ""
echo "Review the plan above."
if terraform show -json tfplan | grep -q '"action":"delete"'; then
  echo "WARNING: This plan includes a DESTROY action."
  read -p "Type 'yes-destroy-prod' to proceed: " confirm
  if [ "$confirm" != "yes-destroy-prod" ]; then
    echo "Aborted."
    exit 1
  fi
fi
terraform apply tfplan
```

This is a simplified stand-in for what Atlantis, Terraform Cloud, or a GitHub Actions approval environment does automatically in a real team setup.

Cleanup

```
# remove prevent_destroy block first if still set
terraform destroy
aws iam detach-role-policy --role-name <your-terraform-execution-role> \
  --policy-arn arn:aws:iam::<account-id>:policy/deny-prod-destroy
aws iam delete-policy --policy-arn arn:aws:iam::<account-id>:policy/deny-prod-destroy
```

Lab Summary Checklist

Lab	What you proved hands-on
1 - State Locking	Second concurrent apply is blocked by DynamoDB lock until the first finishes
2 - Lock File	Provider version stays pinned across re-inits; tampering breaks checksum validation
3A - prevent_destroy	Terraform itself refuses to destroy a protected resource
3B - IAM Deny	AWS rejects the destroy API call regardless of Terraform config
3C - Approval gate	Human confirmation required before any destroy action executes

Note: Run all labs in a sandbox/non-production AWS account. Labs 1 and 3 create real billable resources (EC2 t2.micro — free-tier eligible in most cases, but confirm your account status) — always run the cleanup steps when done.